



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Artificial and Computational Intelligence

AIMLCZG557

ACI Team



AIMLCZG557 – CS#7

Neural Architecture Search (NAS) and Game-Playing (Adversarial Search)

Agenda for CS #7

- 1) Recap of CS #6
- 2) Complete NAS (NEAT and CoDeepNEAT)
- 3) Introduction to Game playing
 - 1) Adversarial Settings?
 - 2) Game tree intuition
 - 3) Types of Games
 - 4) Static evaluation functions
 - 5) Introduction to Minimax algorithm
- 4) Q&A!

Course Plan: Where are we in the Syllabus?



- M1 Introduction to AI
- M2 Problem Solving Agent using Search
- M3 Game Playing
- M4 Knowledge Representation using Logics
- M5 Probabilistic Representation and Reasoning
- M6 Reasoning over time
- M7 Ethics in AI

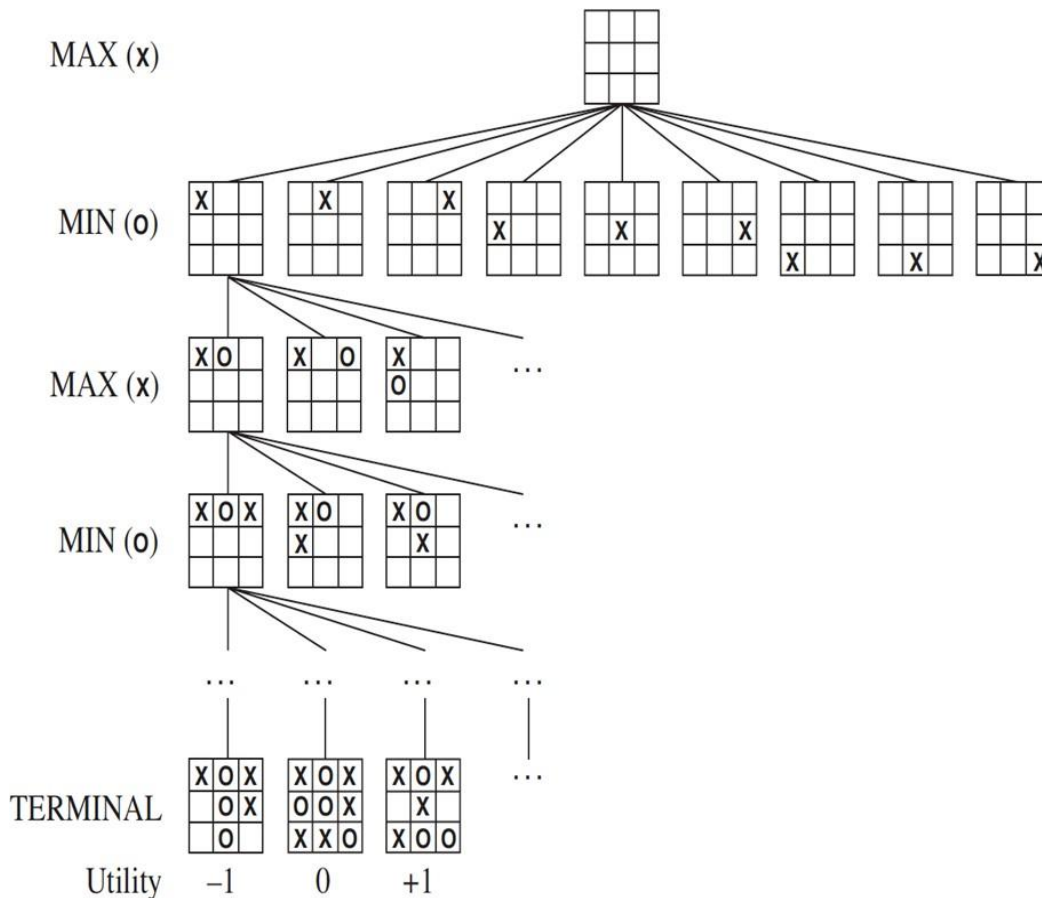
Learning Objectives of M3?

At the end of this class , you should be able to:

1. Convert a given problem into adversarial search problem
2. Formulate the problem solving agent components
3. Design static evaluation function value for a problem
4. Construct a Game tree
5. Apply Min-Max
6. Apply and list nodes pruned by alpha pruning and nodes pruned by beta pruning

Task Environment

Phases of Solution Search by PSA



Assumptions – Environment :

- Static
- Observable
- Discrete
- Deterministic
- Number of Agents

Adversarial Search



- **Adversarial search** is search in a setting where two or more agents have conflicting goals.
- **Adversarial Search** is a branch of artificial intelligence (AI) focused on solving problems where two or more agents (players) have **opposing goals**. In such environments, one player's gain is another's loss.
- In such problems, an agent cannot plan only by asking: “*What action gets me closer to my goal?*”
- It must also ask: “*What will my opponent do to reduce my chance of success?*”
- *Examples: Chess, Tic-Tac-Toe, Go, Checkers, Competitive strategy games*



Normal Search to Adversarial Search

Aspect	Normal Search	Adversarial Search
Number of agents	Usually one	At least two
Goal	Reach a goal state	Win or maximize utility
Environment	Passive	Strategic and competitive
Next state depends on	Our action	Our action and opponent's action
Example	Route finding	Chess

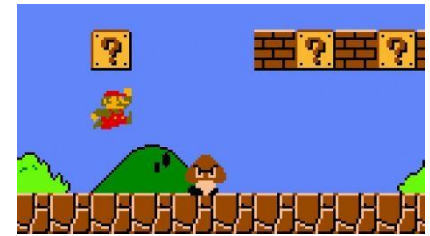
- In normal search, the main issue is finding a path.
- In adversarial search, the main issue is choosing a move while anticipating the opponent's best response.

Game Problem

Study & design of games enables the computers to model ways in which humans think & act hence simulating human intelligence.

AI for Gaming:

- Interesting & Challenging Problem
- Larger Search Space Vs Smaller Solutions
- Explore to better the Human Computer Interaction



Characteristics of Games:

- Observability
- Stochasticity
- Time granularity
- Number of players



Adversarial Games: Goals of agents are in conflict where one's optimized step would reduce the utility value of the other.

A game can be formally described using:

1. Initial state

The starting board or situation.

2. Player function

Tells whose turn it is.

3. Actions function

Gives the legal moves from a state.

4. Result function

Gives the new state after a move.

5. Terminal test

Checks whether the game is over.

6. Utility function

Assigns a final value to the outcome.

Example utility values:

➤ Win = +1

➤ Draw = 0

➤ Loss = -1

MAX wants the score to be as high as possible.

MIN wants the score to be as low as possible.

Game as Search Problem



PSA : Representation of Game:

INITIAL STATE: S_0

PLAYER(s)

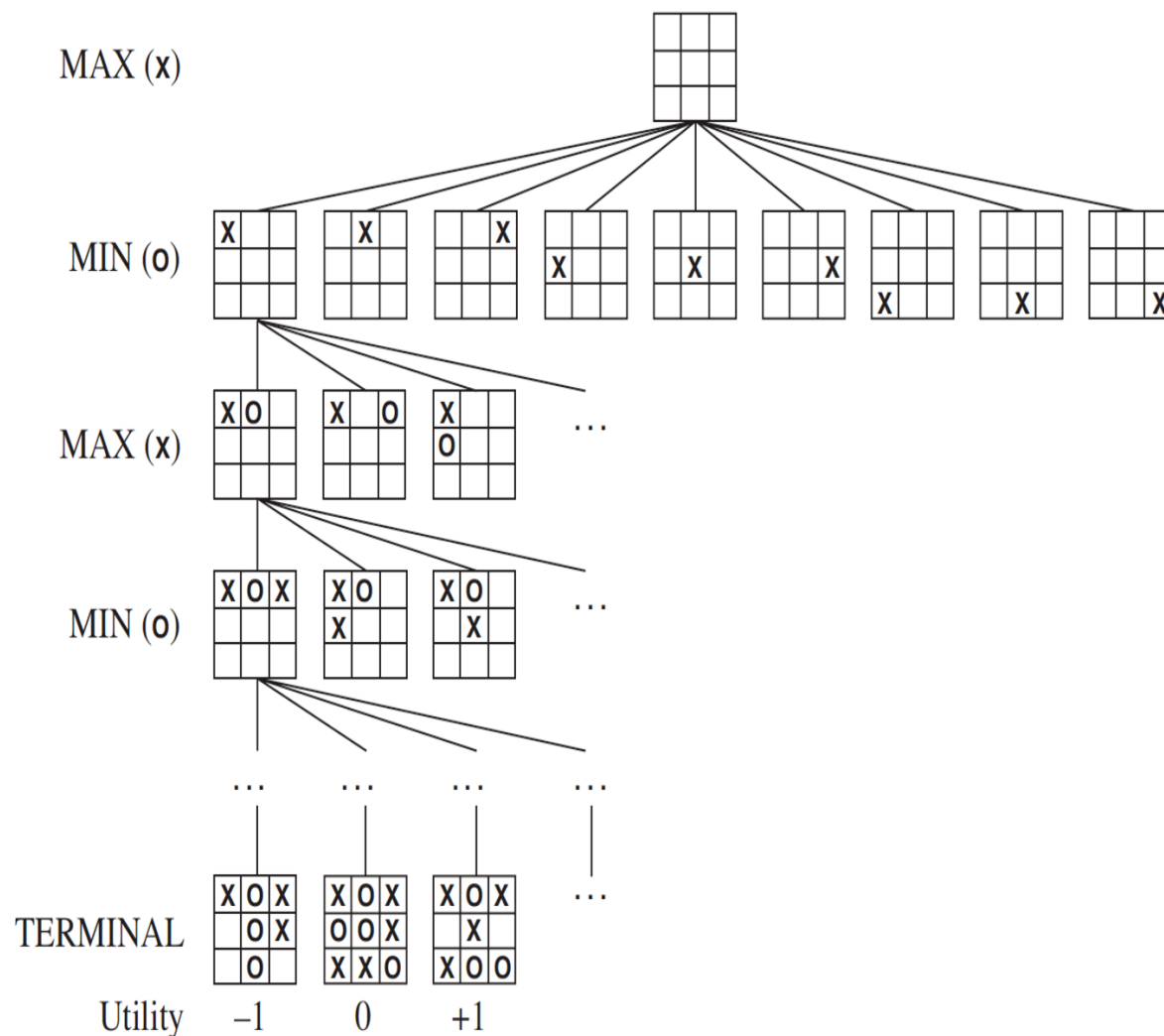
ACTIONS(s)

RESULT(s, a)

TERMINAL-TEST(s)

UTILITY(s, p)

Eg., Tic Tac Toe



Two-Player Zero-Sum Games

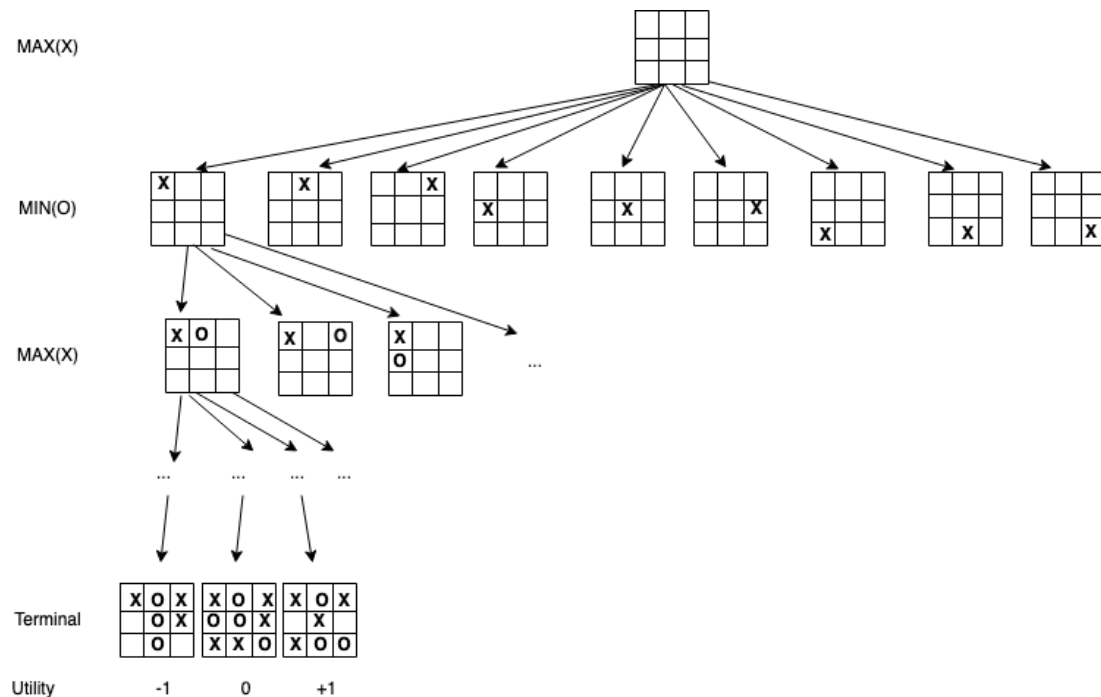


- Many classical AI game-playing examples assume a **two-player zero-sum game**.
- This means:
 - There are two players.
 - One player's gain is the other player's loss.
 - The players have exactly opposite goals.
- We call the players:
 - **MAX**: the player trying to maximize the score.
 - **MIN**: the opponent trying to minimize MAX's score.
- So, if MAX gets +1, MIN effectively gets -1.
- Examples:
 - Tic-tac-toe
 - Chess
 - Checkers

Game Tree



- A **game tree** represents all possible sequences of moves.
- Each level alternates between players.
- At MAX nodes, choose the largest value.
- At MIN nodes, choose the smallest value.
- The values at the leaves represent final outcomes or estimated scores.



Minimax Principle



- The **minimax algorithm** chooses the move that gives the best guaranteed outcome, assuming the opponent also plays optimally.
- The idea:
 - MAX chooses the move with the maximum value.
 - MIN chooses the move with the minimum value.
 - Values are backed up from leaves to the root.
- So, MAX does not simply choose the branch with the best possible outcome.
- MAX chooses the branch whose worst-case outcome is best.

Manual Example: Small Game Tree



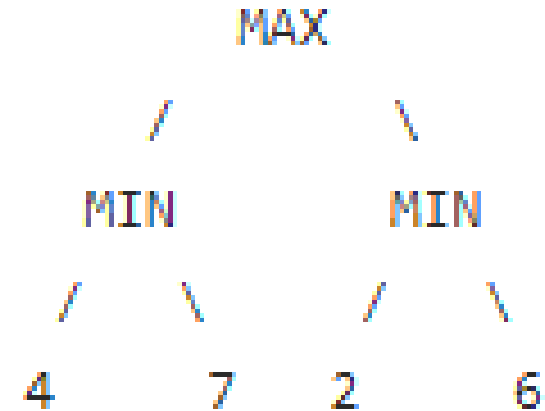
Step 1: Evaluate MIN nodes:

LEFT MIN: $\min(4, 7) = 4$

RIGHT MIN: $\min(2, 6) = 2$

Step 2: Evaluate MAX (root) node:

$\max(4, 2) = 4$



Therefore:

- Best move for MAX: left branch
- Guaranteed value: 4

MAX chooses the left branch because even after MIN's best response, MAX still gets 4.

Minimax Algorithm



- The Minimax algorithm relies on systematic searching, or more accurately said - on **brute force** and a simple evaluation function. Let's assume that every time during deciding the next move we search through a whole tree, all the way down to leaves. Effectively we would look into all the possible outcomes and every time we would be able to determine the best possible move.
- However, for non-trivial games, that practice is inapplicable. Even searching to a certain depth sometimes takes an unacceptable amount of time. Therefore, Minimax applies search to a fairly low tree depth aided with appropriate heuristics, and a well designed, yet simple *evaluation function*.
- With this approach we lose the certainty in finding the best possible move, but the majority of cases the decision that minimax makes is much better than any human's. The principle of minimax is to *minimize the maximum possible loss*.

Minimax Algorithm

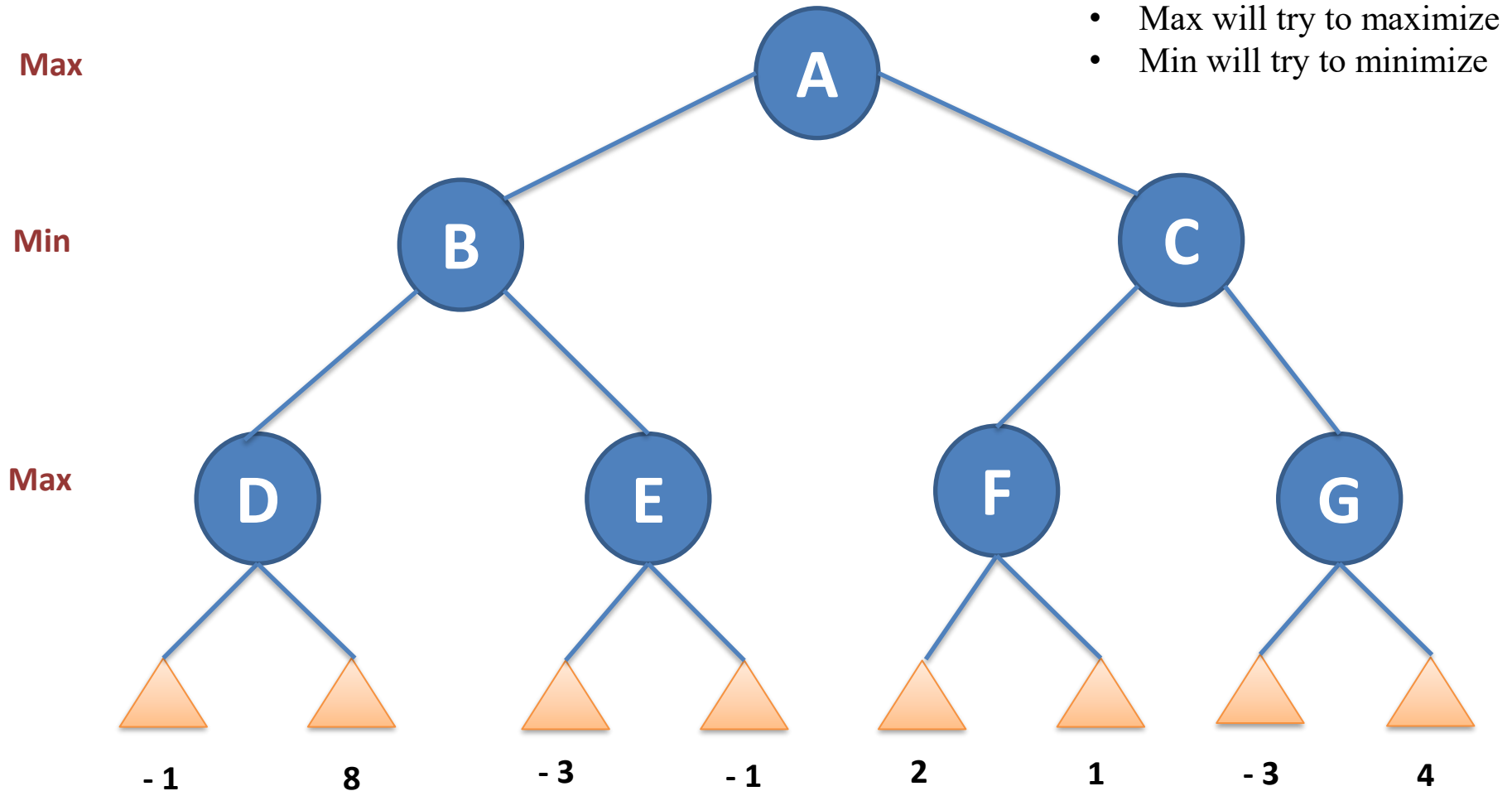


MiniMax Algorithm

Minimax algorithm in Game Theory is one of the oldest algorithms in AI and is used generally for two players. Let's define its rule:

- In this, we assign one player as **Max** who chooses the maximum from the given set of choices
- The other player as **Min** who chooses the minimum from the same collection.
- A **Tree** data structure is used. We designate each level of this tree-structure to both the players Max and Min, alternatively.
- At each level, the decision is made by the player. The last level has all the potential outcomes of the game. In this game, the goal is to reach the top of the tree from the bottom level, without breaking any of the defined rules and eventually solve the problem.

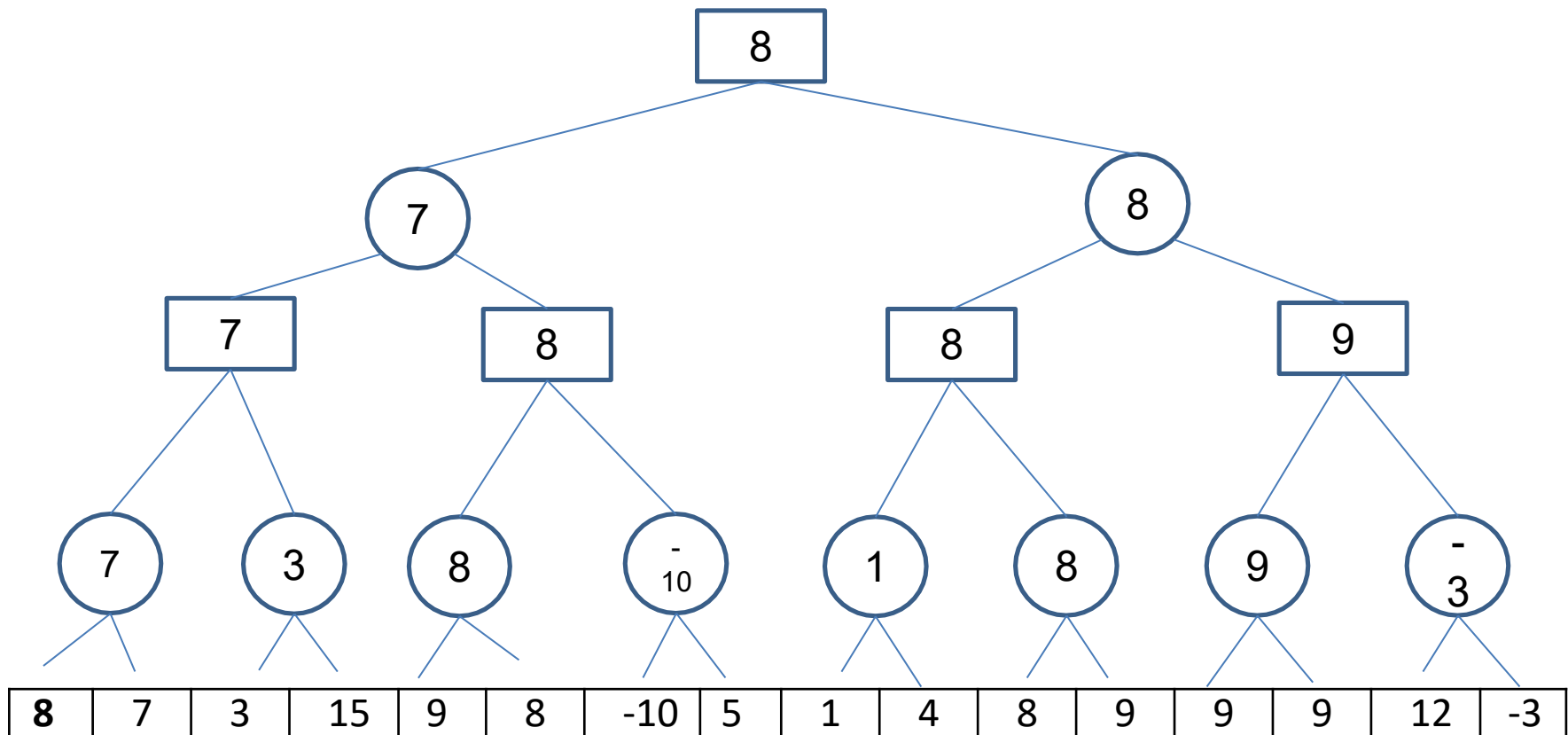
Minimax Algorithm Example



Minimax Algorithm Example 2

Squares represent MAX nodes

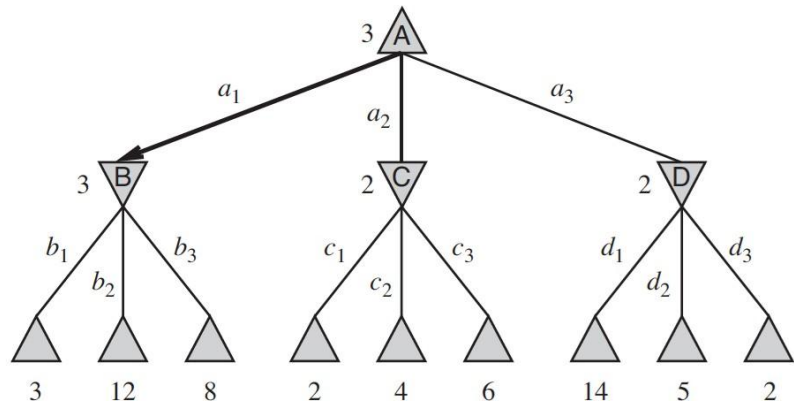
Circles represent MIN nodes



Minimax Algorithm

MAX

MIN



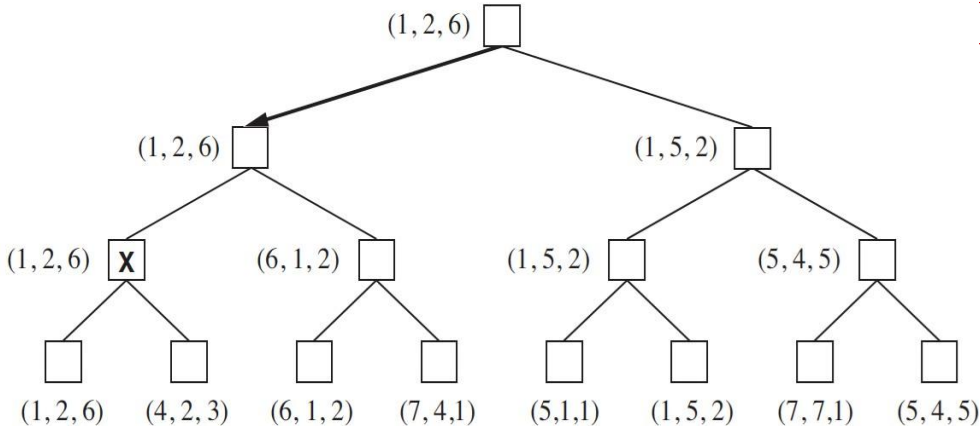
Two Player Game

to move
A

B

C

A



Multiplayer Game

Minimax Algorithm

function MINIMAX-DECISION(*state*) **returns** *an action*
 return $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(state, a))$

function MAX-VALUE(*state*) **returns** *a utility value*
 if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow -\infty$
 for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$
 return *v*

function MIN-VALUE(*state*) **returns** *a utility value*
 if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow \infty$
 for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$
 return *v*

Why Not Search the Whole Tree?



- For small games like tic-tac-toe, full search is possible.
- For large games like chess, the game tree is enormous.
- So, we usually cannot search all the way to terminal states.
- Instead, we use:
 1. A **depth limit**
 2. A **static evaluation function**
- The agent searches a few moves ahead and then estimates how good the resulting state is.

Static Evaluation Function

- A **static evaluation function** estimates the utility of a non-terminal game state.
- It is called “*static*” because it evaluates the board position as it is, without searching further.
- A good evaluation function should:
 - Give higher values to better states for MAX.
 - Give lower values to better states for MIN.
 - Be fast to compute.
 - Correlate reasonably well with the actual chance of winning.
- Example in chess:

Evaluation = material advantage + king safety + mobility + board control

Static Evaluation Function

- Suppose MAX is playing White.

Feature	Meaning	Example score
Material advantage	Does White have more pieces?	+5
King safety	Is White's king safe?	+2
Mobility	Can White make many useful moves?	+1
Board control	Does White control the centre?	+2

- $\text{Evaluation} = 5 + 2 + 1 + 2 = +10$
- So, this position is good for White / MAX.

Simple Evaluation Function Example



- Suppose we use this simple function:

$$\text{Score} = \text{MAX pieces} - \text{MIN pieces}$$

- If MAX has 8 pieces and MIN has 5 pieces:

$$\text{Score} = 8 - 5 = +3$$

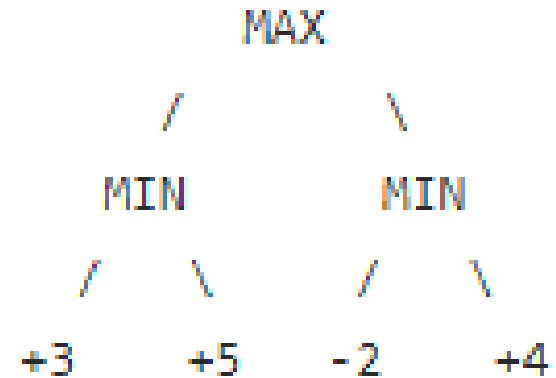
Interpretation:

- Positive score: good for MAX
- Negative score: good for MIN
- Zero: balanced position
- This is not a perfect evaluation, but it gives the agent a useful estimate when full search is not possible.

Manual Game Tree with Static Evaluation



- The leaf values are not final win/loss values. They are estimated values from a static evaluation function.
- **Step 1:** MIN chooses the worse outcome for MAX.
 - Left MIN: $\min(+3, +5) = +3$
 - Right MIN: $\min(-2, +4) = -2$
- **Step 2:** MAX chooses the better guaranteed outcome.
 - $\max(+3, -2) = +3$



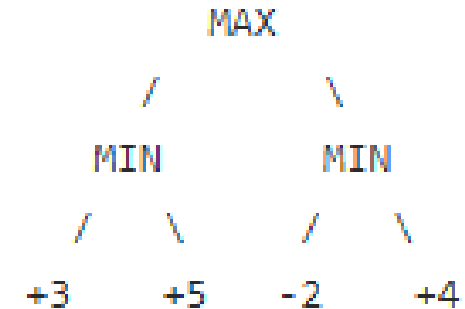
Therefore:

- Best move: left branch
- Backed-up minimax value: +3

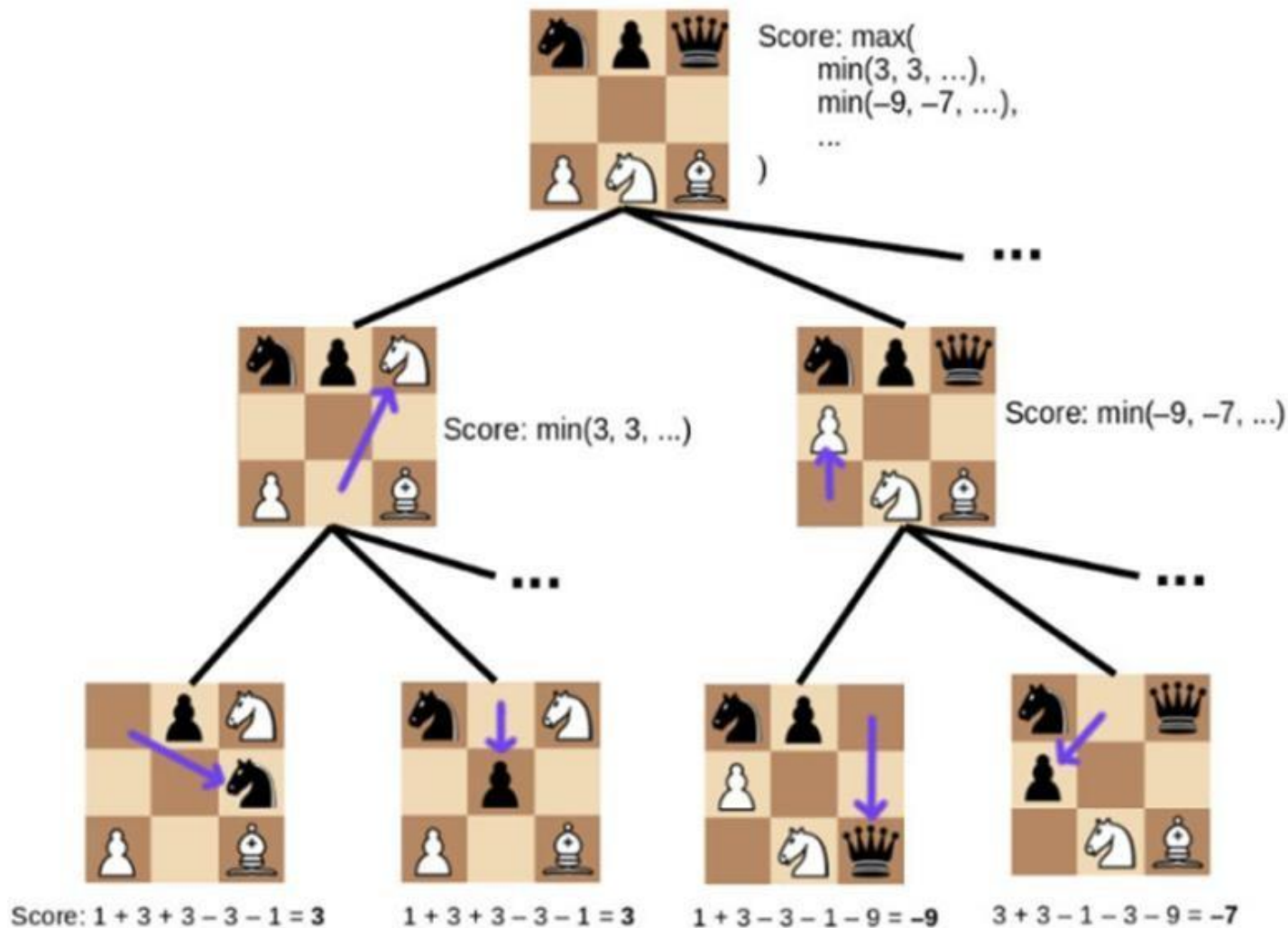
Manual Game Tree with Static Evaluation



- A common mistake students do is to say:
“*MAX should choose the branch that contains +5.*”
- But MAX cannot directly choose +5.
- After MAX chooses a branch, MIN gets a turn!!
- So, if MAX chooses the left branch, MIN will choose +3, not +5.
- That is why the value of the left branch is +3.
- Minimax is about choosing the best move after considering the opponent's best reply!



Design of Static Evaluation Values



Design of Static Evaluation Values



N-Queens

♚			
		♚	♚
	♚		

1	4	2	2	4
---	---	---	---	---

Tic-Tac-Toe

0	0	x
x		0
		x

Max's Share	2
Min's Share	1
Board Value	1

N-Tile

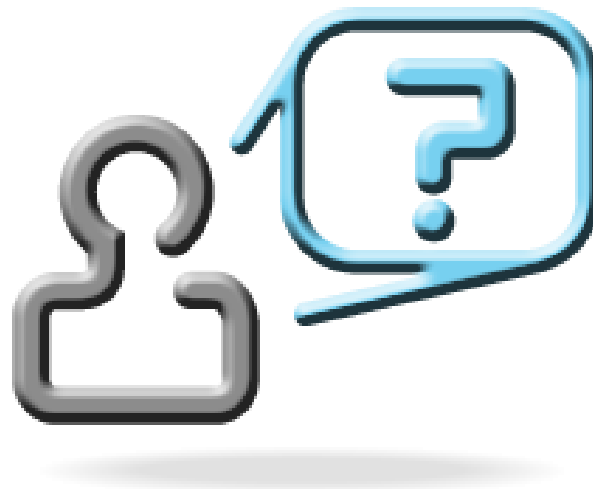
2	8	3
1	6	4
7		5

1	2	3
8		4
7	6	5

No.of.Tiles Out of Place	5
--------------------------	---

$$\text{Eval}(S) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

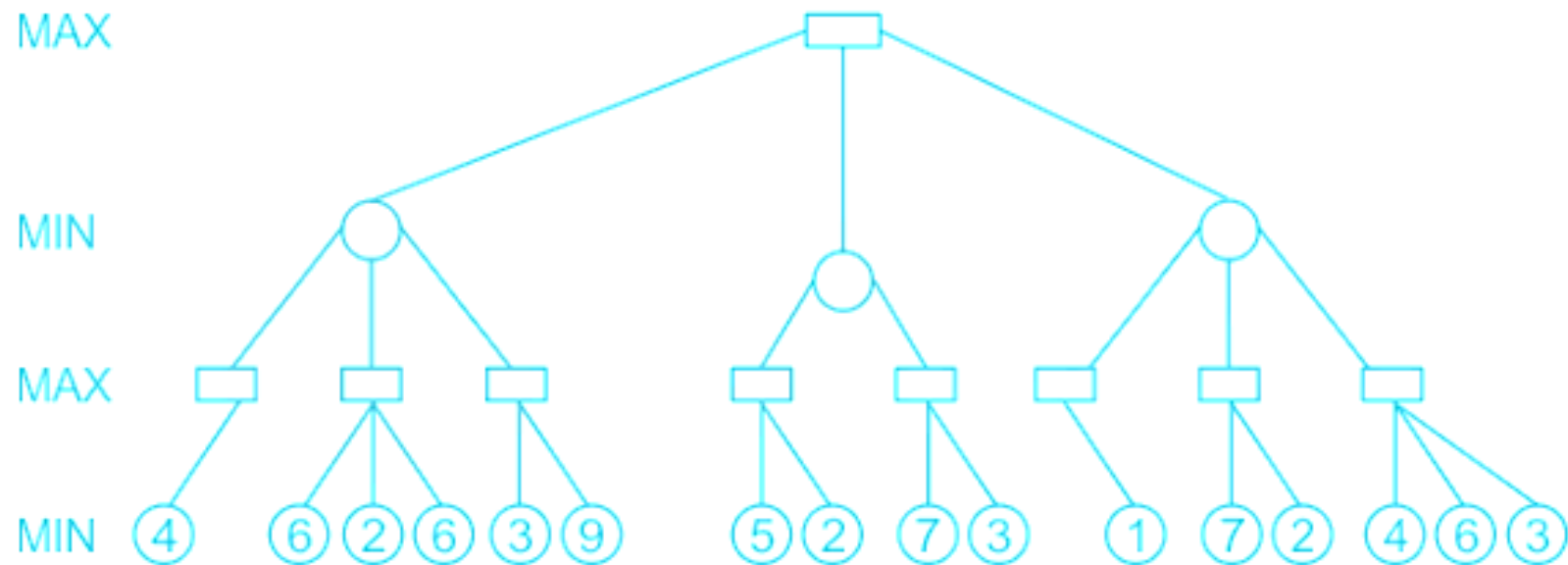
$$= 0.6 (\text{MaxChance} - \text{MinChance}) + 0.4 (\text{MaxPairs} - \text{MinPairs})$$



We will learn more about game playing in the next class ...

Exercise 1

Consider the following minimax game tree search



What will be the value propagated at the root?

Exercise 2: Castle Battle Game

- Two players are playing a small strategy game.
- **MAX** is the attacking army.
- **MIN** is the defending army.
- MAX wants to capture the castle.
- MIN wants to prevent MAX from capturing the castle.
- The game is not fully played until the end. We will search only **2 moves ahead** and then use a **static evaluation function**.

Part A: Draw the Game Tree

At the current position, MAX has three possible moves:

1. **Attack Left Gate**
2. **Attack Main Gate**
3. **Attack Right Gate**

After each MAX move, MIN has two possible replies.

MAX move	MIN reply 1	MIN reply 2
Attack Left Gate	Reinforce Left Gate	Counterattack Left
Attack Main Gate	Reinforce Main Gate	Block Supply Route
Attack Right Gate	Reinforce Right Gate	Counterattack Right

Exercise 2: Castle Battle Game

Task 1

- Draw the game tree up to depth 2.
- The root node should be MAX's turn.
- The second level should be MIN's turn.
- The leaf nodes should be the board positions after MIN replies.

Part B: Design a Static Evaluation Function

At each leaf node, the game is not over. So, we need to estimate how good the position is for MAX.

Assume we judge the position using four features:

Feature	Meaning
Soldier advantage	MAX soldiers – MIN soldiers
Gate damage	Damage done to castle gates
MAX safety	How safe MAX's army is
Supply control	Whether MAX controls the supply route

$$\text{Evaluation} = 2 \times \text{Soldier Advantage} + 3 \times \text{Gate Damage} + 2 \times \text{MAX Safety} + 4 \times \text{Supply Control}$$

Exercise 2: Castle Battle Game

Leaf	Soldier Advantage	Gate Damage	MAX Safety	Supply Control
L1	2	3	2	0
L2	1	4	0	1
M1	3	2	3	0
M2	1	3	1	2
R1	2	2	2	1
R2	0	5	0	1

Given the above feature values (leaf), compute the evaluation scores using the eval function.

ACI: Disclaimer and Acknowledgements

- Few content for these slides may have been obtained from prescribed books and various other source(s) on the Internet.
- We hereby acknowledge all the contributors for their material and inputs and gratefully acknowledge people others who made their course materials freely available online. We have provided source information wherever necessary.
- We have added and modified the content to suit the requirements of the class dynamics & live session's lecture delivery flow for presentation.
- This *is not a full-fledged* reading materials. Students are requested to refer to the textbook w.r.t detailed content as per the course handout as shared over e-learning portal - Taxila.
- **Slide Source / Preparation / Review:**
- From BITS Pilani WILP: Prof. Rajavadhana, Prof. Indumathi, Prof. Sangeetha and Prof Parthasarathy PD, Prof Ramya Devi
- From BITS On-campus & External : Mr.Santosh GSK

Thank You for your
time & attention !

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)

